

Portfolio

Manual Packet Filtering Firewalls (IPTABLES)

Prepared by: Mubeen Khan

Date: 04/09/2025

Table of Contents

1.0 Introduction

- 1.1 Portfolio Objectives
- 1.2 Lab Environment Setup

2.0 Exercise 1: Installation and Initial Configuration

- 2.1 Installing IPTABLES
- 2.2 Accessing the IPTABLES Manual
- 2.3 Viewing Default Rules and Configuration

3.0 Exercise 2: Implementing Firewall Rules

- 3.1 Blocking a Specific IP Address
- 3.2 Unblocking a Specific IP Address
- 3.3 Blocking Outgoing Connections on a Specific Port
- 3.4 Allowing Incoming Connections on a Specific Port
- 3.5 Allowing Multiple Ports Using Multiport
- 3.6 Blocking Access to a Network (Facebook)
- 3.7 Mitigating Network Flood on a Service Port (Apache)
- 3.8 Blocking Incoming ICMP (Ping) Requests
- 3.9 Allowing Loopback Access
- 3.10 Logging Dropped Packets
- 3.11 Blocking Access by MAC Address
- 3.12 Limiting Concurrent Connections per IP
- 3.13 Searching Within Existing Rules
- 3.14 Flushing (Resetting) All Rules
- 3.15 Blocking Outgoing SMTP Mail

4.0 Challenges

- 4.1 Common Issues Faced

5.0 Conclusion

1.0 Introduction

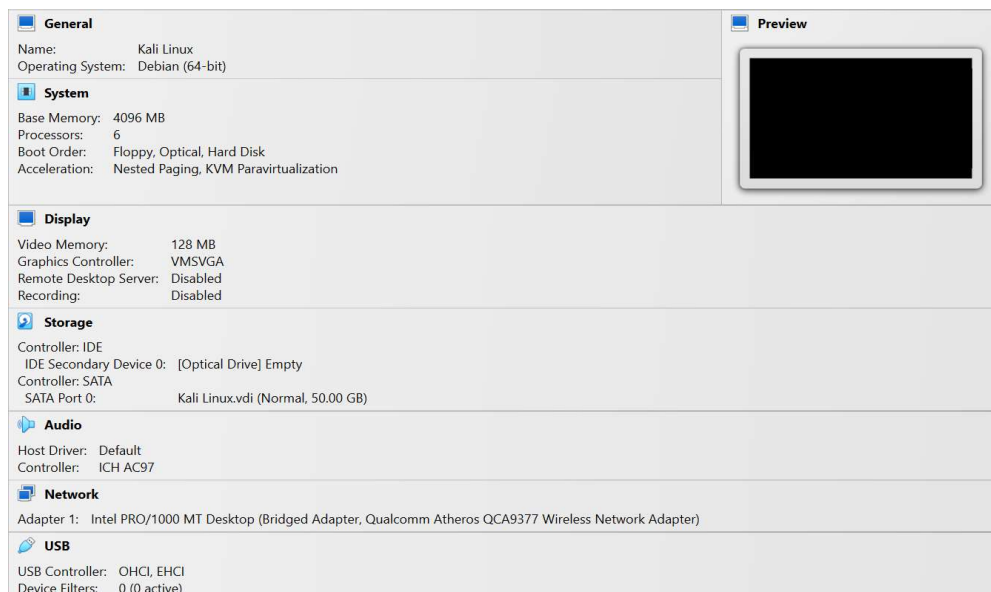
1.1 Portfolio Objective

The primary objective of this portfolio is to understand and carry out practical implementation of network security principles by configuring a manual packet-filtering firewall. We will be using IPTABLES firewall on Kali-Linux to gain familiarity with IPTABLES firewalls, explore the functionalities of IPTABLES, with Linux firewall, implement security policy, apply advanced filtering techniques, and develop troubleshooting skills.

1.2 Lab Environment Setup

A controlled and isolated virtual environment was established using Oracle VirtualBox to ensure the lab exercises could be conducted safely without impacting any external networks or systems.

- **Virtualization Platform: Oracle VirtualBox** (Version 7.0 or later).
- **Virtual Machine (VM) Configuration:**
 - **Name:** Kali Linux
 - **Operating System:** Debian (64-bit)
 - **Memory (RAM):** 4096 MB
 - **Processors (CPU):** 6
 - **Storage:** A dynamically allocated virtual hard disk (Kali Linux.vdi) with a maximum size of 50.00 GB.

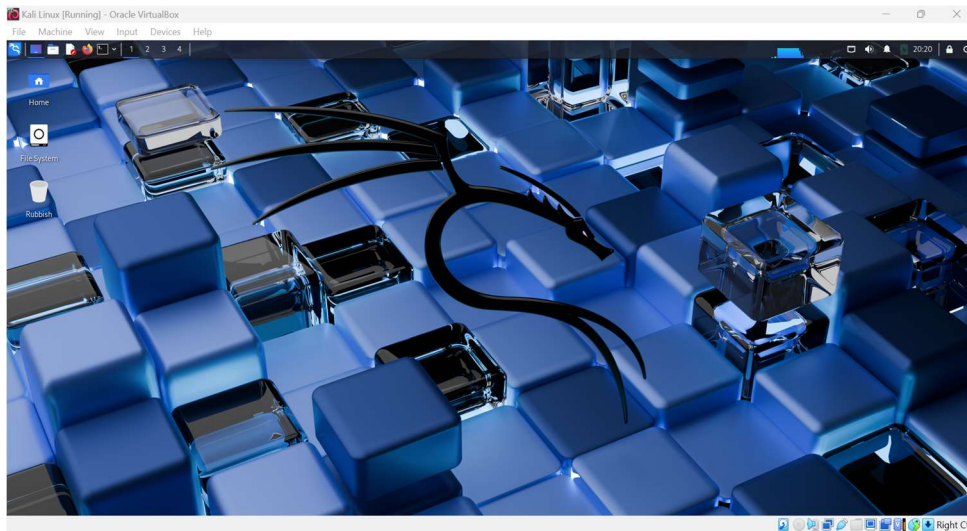


- **Network Adapter Configuration:**

- **Mode:** Bridged Adapter
- **Reason for Choice:** This mode provides the most realistic testing environment by placing the VM directly on the local network as a distinct device. It allows for comprehensive testing of both **inbound** (e.g., ping scans, connection attempts) and **outbound** (e.g., blocking social media sites) firewall rules from other machines, which is crucial for this lab.

Comparison to NAT Mode: While NAT mode provides internet access, it hides the VM behind the host's IP address, preventing other devices from initiating connections to it. This makes it unsuitable for testing inbound filtering rules, a core objective of this exercise. Bridged mode offers the ideal blend of external accessibility and full internet connectivity.

My Linux Interface



2.0 Exercise 1: Installation and Initial Configuration

2.1 Installing IPTABLES

The first step was to ensure the iptables firewall utility was installed on the Kali Linux system. Although it is typically pre-installed on security-focused distributions like Kali, the installation command was run to confirm and ensure the latest version was present. We used the command “sudo apt-get install iptables”.

```
mubeen@kali: ~  
File Actions Edit View Help  
(mubeen@kali)-[~]  
$ sudo apt-get install iptables  
[sudo] password for mubeen:  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
iptables is already the newest version (1.8.11-2).  
0 upgraded, 0 newly installed, 0 to remove and 461 not upgraded.  
(mubeen@kali)-[~]  
$
```

2.2 Accessing the IPTABLES Manual

To understand the extensive syntax, options, and capabilities of the iptables command, the manual page was accessed. This is an essential reference for constructing correct and effective rules. I used the command “man iptables”

```
mubeen@kali: ~  
File Actions Edit View Help  
IPTABLES(8) iptables 1.8.11 IPTABLES(8)  
NAME  
iptables/ip6tables - administration tool for IPv4/IPv6 packet filtering and NAT  
SYNOPSIS  
iptables [-t table] {-A|-C|-D|-V} chain rule-specification  
ip6tables [-t table] {-A|-C|-D|-V} chain rule-specification  
iptables [-t table] -I chain [rulenum] rule-specification  
iptables [-t table] -R chain rulenum rule-specification  
iptables [-t table] -D chain rulenum  
iptables [-t table] -S [chain [rulenum]]  
iptables [-t table] {-F|-L|-Z} [chain [rulenum]] [options...]  
iptables [-t table] -N chain  
iptables [-t table] -X [chain]  
iptables [-t table] -P chain policy  
Manual page iptables(8) line 1 (press h for help or q to quit)
```

2.3 Viewing default Rules and Configuration

Before implementing any custom rules, it was crucial to inspect the default firewall configuration to establish a baseline. The -L (list), -n (numeric output), and -v (verbose) flags were used to display the current rules in a clear and detailed format.

```
mubeen@kali: ~  
File Actions Edit View Help  
(mubeen@kali)-[~]  
$ sudo apt-get install iptables  
[sudo] password for mubeen:  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
iptables is already the newest version (1.8.11-2).  
0 upgraded, 0 newly installed, 0 to remove and 461 not upgraded.  
(mubeen@kali)-[~]  
$ man iptables  
(mubeen@kali)-[~]  
$ sudo iptables -L -n -v  
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)  
  pkts bytes target     prot opt in     out     source destination  
on  
Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)  
  pkts bytes target     prot opt in     out     source destination  
on  
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)  
  pkts bytes target     prot opt in     out     source destination  
on  
(mubeen@kali)-[~]  
$
```

This output shows that all three default chains (INPUT, FORWARD, OUTPUT) have a default **policy** of ACCEPT. This means that, by default, the firewall allows all incoming, outgoing, and forwarded traffic because no rules exist to deny it. The goal of the lab is to move from this open state to a controlled one by adding specific deny and allow rules.

3.0 Implementing Firewall Rules

3.1 Blocking a specific IP Address

The goal of this rule was to deny all incoming network traffic originating from a specific source IP address, effectively blocking that machine from communicating with our system. We used the command “`sudo iptables -A INPUT -s 192.168.1.100 -j DROP`”, where it then asked for my password to verify and apply the command.

- “-A INPUT”: Appends this rule to the end of the INPUT chain.
- “-s 192.168.1.100”: specifies the source (-s) IP address to match.
- “-j DROP”: it indicates that the matching packets should be dropped silently.

We also then verified that the rule has successfully been applied using the command “`sudo iptables -L INPUT -n -v`”.

```
mubeen@kali: ~  
File Actions Edit View Help  
(mubeen@kali)-[~]  
$ sudo iptables -L -n -v  
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)  
  pkts bytes target     prot opt in     out     source destination  
on  
Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)  
  pkts bytes target     prot opt in     out     source destination  
on  
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)  
  pkts bytes target     prot opt in     out     source destination  
on  
(mubeen@kali)-[~]  
$ sudo iptables -A INPUT -s 192.168.1.100 -j DROP  
[sudo] password for mubeen:  
(mubeen@kali)-[~]  
$ sudo iptables -L INPUT -n -v  
Chain INPUT (policy ACCEPT 11 packets, 1980 bytes)  
  pkts bytes target     prot opt in     out     source destination  
on  
  0      0 DROP      all  --  *      *       192.168.1.100  0.0.0.0/0  
(mubeen@kali)-[~]  
$
```

3.2 Unblocking a specific IP Address

This command removed the previously created blocking rule, demonstrating how to manage and alter the firewall ruleset dynamically.

“-D” command is used to delete the specific rule.

```
(mubeen@kali)-[~]
$ sudo iptables -D INPUT -s 192.168.1.100 -j DROP

(mubeen@kali)-[~]
$ sudo iptables -L INPUT -n -v
Chain INPUT (policy ACCEPT 199 packets, 36377 bytes)
 pkts bytes target    prot opt in     out     source            destination
 0      0 DROP      tcp    --    *      *       0.0.0.0/0         0.0.0.0/0
    tcp dpt:8080

(mubeen@kali)-[~]
$
```

3.3 Blocking Outgoing Connections on a Specific Port

This rule was designed to prevent any outgoing traffic from the system to TCP port 8080, which is often used for HTTP proxy or web caching services.

I used command “sudo iptables -A OUTPUT -p tcp --dport 8080 -j DROP”

- -A output: Appends the rule to OUTPUT chain.
- -p tcp: specifies the protocol (-p) as TCP.
- --dport 8080: Matches packets where the destination port (--dport) is 8080

Then I verified that the rule/command has been successfully applied using “sudo iptables -L OUTPUT -n -v”.

```
(mubeen@kali)-[~]
$ sudo iptables -A OUTPUT -p tcp --dport 8080 -j DROP

(mubeen@kali)-[~]
$ sudo iptables -L OUTPUT -n -v
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source            destination
 0      0 DROP      tcp    --    *      *       0.0.0.0/0         0.0.0.0/0
    tcp dpt:8080

(mubeen@kali)-[~]
$
```

3.4 Allowing Incoming Connections on a Specific Port

This rule demonstrates a fundamental principle of firewall configuration: explicitly allowing essential services. Here, we ensure that incoming SSH connections (TCP port 22) are permitted, which is crucial for remote administration of the server.

```
(mubeen@kali)-[~]
$ sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT

(mubeen@kali)-[~]
$ sudo iptables -L INPUT -n -v
Chain INPUT (policy ACCEPT 462 packets, 83856 bytes)
 pkts bytes target     prot opt in     out     source         destination
  0      0 ACCEPT     tcp  --  *      *       0.0.0.0/0      0.0.0.0/0
cp dpt:22

(mubeen@kali)-[~]
$
```

Note: TCP can be replaced with UDP if you want to allow/blocking incoming/outgoing traffic from UDP connections.

3.5 Allowing Multiple Ports using Multiport

This rule demonstrates an efficient method for managing firewall rules by grouping multiple related ports into a single, streamlined rule. This improves readability and performance by reducing the total number of rules the firewall needs to process. Here, we allow incoming traffic for standard web services: HTTP (port 80) and HTTPS (port 443).

For Multiport we used the command “sudo iptables -A INPUT -p tcp -m multiport --dports 80,443 -j ACCEPT”

- -A INPUT: Appends the rule to the INPUT chain.
- -p tcp: Specifies TCP protocol.
- -m multiport: Loads the multiport matching module (-m), which allows for matching multiple ports in a single rule.
- --dports 80, 443: Used for defining the destination ports: 80 and 443.
- -j ACCEPT: Accepts any matching making traffic.

```
(mubeen@kali)-[~]
$ sudo iptables -A INPUT -p tcp -m multiport --dports 80,443 -j ACCEPT

(mubeen@kali)-[~]
$ sudo iptables -L INPUT -n -v
Chain INPUT (policy ACCEPT 1046 packets, 190K bytes)
 pkts bytes target     prot opt in     out     source         destination
  0      0 ACCEPT     tcp  --  *      *       0.0.0.0/0      0.0.0.0/0
cp dpt:22
  0      0 ACCEPT     tcp  --  *      *       0.0.0.0/0      0.0.0.0/0
multiport dports 80,443

(mubeen@kali)-[~]
$
```

3.6 Blocking Facebook on IPTABLES Firewall

This rule demonstrates how to enforce an access control policy by blocking traffic to a specific external service. The goal is to prevent the system from communicating with Facebook's network.

Step 1: Find Facebook's IP Address(es)

First, we used “nslookup” to resolve the domain name facebook.com to its current IP

addresses. This is a crucial step because domain names can point to multiple, changing IP addresses.

```
(mubeen@kali)-[~]
$ nslookup facebook.com
Server:      192.168.20.1
Address:     192.168.20.1#53

Non-authoritative answer:
Name:   facebook.com
Address: 157.240.8.35
Name:   facebook.com
Address: 2a03:2880:f119:8083:face:b00c:0:25de

(mubeen@kali)-[~]
$
```

The output shows that facebook.com resolves to the IPv4 address 157.240.229.35 (and an IPv6 address). This is the IP we will target.

Step 2: Block Outgoing Traffic to Facebook's IP

We added a rule to the OUTPUT chain to drop any packets destined for the IP address we found.

```
(mubeen@kali)-[~]
$ sudo iptables -A OUTPUT -d 157.240.8.35 -j DROP

(mubeen@kali)-[~]
$ sudo iptables -L OUTPUT -n -v
Chain OUTPUT (policy ACCEPT 445 packets, 47074 bytes)
pkts bytes target      prot opt in      out     source       destination
  0    0 DROP        tcp  --  *       *       0.0.0.0/0    0.0.0.0/0
cp dpt:8080
  0    0 DROP        all  --  *       *       0.0.0.0/0    157.240.8.35

(mubeen@kali)-[~]
$
```

Note: This method blocks a single IP. In a real-world scenario, Facebook uses a large range of IPs. A more robust implementation would block an entire subnet (e.g., 157.240.229.0/24) or use a more advanced method like DNS filtering.

3.7 Block Network Flood on Apache Port with IPTABLES

This rule demonstrates how to use iptables for rate-limiting to protect a network service from being overwhelmed by too many connection requests in a short period. This is a basic but effective measure against flood attacks, such as a DoS attack targeting a web server.

```
(mubeen@kali)-[~]
$ sudo iptables -A INPUT -p tcp --dport 80 -m limit --limit 100/min --limit-burst 200 -j ACCEPT

(mubeen@kali)-[~]
$ sudo iptables -L INPUT -n -v
Chain INPUT (policy ACCEPT 1443 packets, 262K bytes)
pkts bytes target      prot opt in      out     source    destination
0      0 ACCEPT      tcp  --  *      *       0.0.0.0/0  0.0.0.0/0    t
cp dpt:22
0      0 ACCEPT      tcp  --  *      *       0.0.0.0/0  0.0.0.0/0    m
ulthood dports 80,443
0      0 ACCEPT      tcp  --  *      *       0.0.0.0/0  0.0.0.0/0    t
cp dpt:80 limit: avg 100/min burst 200
```

- - m limit: Loads the limit match module, which is used to control the rate of matches.
- --limit 100/min: sets the average maximum rate. This allows 100 new connections per minute on port 80. Once the limit is hit, further packets will be evaluated by the next rule in chain.
- --limit-burst 200: Sets the initial burst threshold. The first 200 connection packets are allowed immediately. The bucket is then replenished at the rate of 100/min. This allows for normal traffic bursts.

3.8 Blocking Incoming Ping Requests on IPTABLES

This rule demonstrates how to control ICMP (Internet Control Message Protocol) traffic, specifically to enhance security and privacy by preventing the system from responding to external ping requests. This makes the host less visible and harder to detect on a network.

```
(mubeen@kali)-[~]
$ sudo iptables -A INPUT -p icmp --icmp-type echo-request -j DROP

(mubeen@kali)-[~]
$ sudo iptables -L INPUT -n -v
Chain INPUT (policy ACCEPT 1599 packets, 291K bytes)
pkts bytes target      prot opt in      out     source    destination
0      0 ACCEPT      tcp  --  *      *       0.0.0.0/0  0.0.0.0/0    t
cp dpt:22
0      0 ACCEPT      tcp  --  *      *       0.0.0.0/0  0.0.0.0/0    m
ulthood dports 80,443
0      0 ACCEPT      tcp  --  *      *       0.0.0.0/0  0.0.0.0/0    t
cp dpt:80 limit: avg 100/min burst 200
0      0 DROP       icmp --  *      *       0.0.0.0/0  0.0.0.0/0    i
cmptype 8
```

- -p icmp: Specifies the ICMP protocol.
- --icmp-type echo-request: Matches the ICMP “Echo Request” packet type which is used by the ping command.

3.9 Allowing Loopback Access

This rule ensures that internal communication within the local machine itself remains functional. The loopback interface (lo) is a virtual network interface that the system uses to

communicate with itself. Many critical local services and applications depend on this internal traffic, so it must always be permitted.

```
(mubeen@kali)-[~]
$ sudo iptables -A INPUT -i lo -j ACCEPT

(mubeen@kali)-[~]
$ sudo iptables -A OUTPUT -o lo -j ACCEPT

(mubeen@kali)-[~]
$ sudo iptables -L INPUT -n -v
Chain INPUT (policy ACCEPT 1775 packets, 323K bytes)
pkts bytes target      prot opt in     out    source        destination
 0      0 ACCEPT      tcp  --  *      *       0.0.0.0/0      0.0.0.0/0      t
cp dpt:22
 0      0 ACCEPT      tcp  --  *      *       0.0.0.0/0      0.0.0.0/0      m
ultiport dports 80,443
 0      0 ACCEPT      tcp  --  *      *       0.0.0.0/0      0.0.0.0/0      t
cp dpt:80 limit: avg 100/min burst 200
 0      0 DROP        icmp --  *      *       0.0.0.0/0      0.0.0.0/0      i
cmptype 8
 0      0 ACCEPT      all  --  lo     *       0.0.0.0/0      0.0.0.0/0
(mubeen@kali)-[~]
$
```

The rule was successfully added to the INPUT and OUTPUT chain. The firewall will now accept all incoming traffic on the loopback interface. The packet (pkts) and byte (bytes) count show that traffic has already been processed by this rule, confirming that internal services are actively using the loopback interface.

3.10 Keep a Log of Dropped Network Packets

This rule demonstrates how to enable logging for diagnostic purposes. It instructs the firewall to create a log entry in the system log (/var/log/syslog) for every packet that is dropped by the INPUT chain. This is invaluable for troubleshooting connectivity issues, identifying attack patterns, and auditing firewall performance.

```
(mubeen@kali)-[~]
$ sudo iptables -A INPUT -j LOG --log-prefix "IPTABLES-DROPPED:" --log-level 4

(mubeen@kali)-[~]
$ sudo iptables -L INPUT -n -v
Chain INPUT (policy ACCEPT 1879 packets, 341K bytes)
pkts bytes target      prot opt in     out    source        destination
 0      0 ACCEPT      tcp  --  *      *       0.0.0.0/0      0.0.0.0/0      t
cp dpt:22
 0      0 ACCEPT      tcp  --  *      *       0.0.0.0/0      0.0.0.0/0      m
ultiport dports 80,443
 0      0 ACCEPT      tcp  --  *      *       0.0.0.0/0      0.0.0.0/0      t
cp dpt:80 limit: avg 100/min burst 200
 0      0 DROP        icmp --  *      *       0.0.0.0/0      0.0.0.0/0      i
cmptype 8
 0      0 ACCEPT      all  --  lo     *       0.0.0.0/0      0.0.0.0/0
 2    634 LOG        all  --  *      *       0.0.0.0/0      0.0.0.0/0      L
LOG flags 0 level 4 prefix "IPTABLES-DROPPED:"
(mubeen@kali)-[~]
$
```

- **-j LOG:** The "jump" target to the LOG module, which generates a kernel log for the packet.
- **--log-prefix "IPTABLES-DROPPED: ":** Adds a custom prefix to all log messages generated by this rule. This makes it easy to grep (search) for these specific entries in the log files.

- `--log-level 4`: Sets the syslog level to "Warning". This is a common level for security-related messages.

3.11 Blocking Access to Specific MAC Address on IPTABLES

This rule demonstrates how to filter network traffic based on hardware addresses. A MAC address is a unique identifier assigned to a network interface controller (NIC). Blocking by MAC address is a stringent security measure because it restricts access based on the physical device itself, regardless of the IP address it might be using. This is useful for blacklisting a specific known malicious device on a local network.

```
(mubeen@kali)-[~]
$ sudo iptables -A INPUT -m mac --mac-source 00:1A:2B:3C:4D:5E -j DROP

(mubeen@kali)-[~]
$ sudo iptables -L INPUT -n -v
Chain INPUT (policy ACCEPT 2012 packets, 365K bytes)
pkts bytes target prot opt in out source destination
0 0 ACCEPT tcp -- * * 0.0.0.0/0 0.0.0.0/0 t
cp dpt:22
0 0 ACCEPT tcp -- * * 0.0.0.0/0 0.0.0.0/0 m
ulthood dports 80,443
0 0 ACCEPT tcp -- * * 0.0.0.0/0 0.0.0.0/0 t
cp dpt:80 limit: avg 100/min burst 200
0 0 DROP icmp -- * * 0.0.0.0/0 0.0.0.0/0 i
cmptype 8
0 0 ACCEPT all -- lo * 0.0.0.0/0 0.0.0.0/0
135 24589 LOG all -- * * 0.0.0.0/0 0.0.0.0/0 L
OG flags 0 level 4 prefix "IPTABLES-DROPPED:"
0 0 DROP all -- * * 0.0.0.0/0 0.0.0.0/0 M
AC 00:1a:2b:3c:4d:5e

(mubeen@kali)-[~]
$
```

3.12 Limiting the Number of Concurrent Connections per IP Address

This rule demonstrates how to use connection tracking to protect a service from being overwhelmed by too many simultaneous connections from a single source IP address. This is an effective defense against brute-force attacks (e.g., on SSH) and Denial-of-Service (DoS) attempts that aim to consume all available server resources.

```
(mubeen@kali)-[~]
$ sudo iptables -A INPUT -p tcp --syn --dport 22 -m connlimit --connlimit-above 3 -j REJECT

(mubeen@kali)-[~]
$ sudo iptables -L INPUT -n -v
Chain INPUT (policy ACCEPT 2105 packets, 383K bytes)
pkts bytes target prot opt in out source destination
0 0 ACCEPT tcp -- * * 0.0.0.0/0 0.0.0.0/0 t
cp dpt:22
0 0 ACCEPT tcp -- * * 0.0.0.0/0 0.0.0.0/0 m
ulthood dports 80,443
0 0 ACCEPT tcp -- * * 0.0.0.0/0 0.0.0.0/0 t
cp dpt:80 limit: avg 100/min burst 200
0 0 DROP icmp -- * * 0.0.0.0/0 0.0.0.0/0 i
cmptype 8
0 0 ACCEPT all -- lo * 0.0.0.0/0 0.0.0.0/0
228 41715 LOG all -- * * 0.0.0.0/0 0.0.0.0/0 L
OG flags 0 level 4 prefix "IPTABLES-DROPPED:"
0 0 DROP all -- * * 0.0.0.0/0 0.0.0.0/0 M
AC 00:1a:2b:3c:4d:5e
0 0 REJECT tcp -- * * 0.0.0.0/0 0.0.0.0/0 t
cp dpt:22 flags:0x17/0x02 #conn src/32 > 3 reject-with icmp-port-unreachable

(mubeen@kali)-[~]
$
```

- -p tcp --syn --dport 22: Matches TCP packets with the SYN flag set (indicating a new connection request) destined for port 22 (SSH).
- -m connlimit: Loads the connlimit (connection limit) matching module.
- --connlimit-above 3: Triggers a match if the source IP address already has **more than 3** established connections. This limits each client IP to a maximum of 3 concurrent SSH sessions.
- -j REJECT: **Rejects** matching packets, sending a "connection refused" error back to the source. This is often more polite than DROP for this scenario, as it immediately informs the client they are being rate-limited.

3.13 Search within IPTABLES Rules

As a firewall ruleset grows in complexity, the ability to quickly search for and locate specific rules becomes essential for management, auditing, and troubleshooting.

```
(mubeen@kali)-[~]
$ sudo iptables-save | grep "22"
-A INPUT -p tcp -m tcp --dport 22 -j ACCEPT
-A INPUT -p tcp -m tcp --dport 22 --tcp-flags FIN,SYN,RST,ACK SYN -m connlimit --connlimit
above 3 --connlimit-mask 32 --connlimit-saddr -j REJECT --reject-with icmp-port-unreachab
le
(mubeen@kali)-[~]
$
```

- sudo iptables-save: This command dumps the entire, current iptables ruleset from all tables (filter, nat, mangle) to stdout. The output is presented in the form of commands that can be used to restore the rules.
- | grep "22": The pipe (|) sends the output of iptables-save to the grep command, which filters the output and prints only the lines that contain the string "22" (e.g., looking for rules related to port 22).

Another example for searching "DROP" IPTABLE:

```
(mubeen@kali)-[~]
$ sudo iptables -S | grep "DROP"
-A INPUT -p icmp -m icmp --icmp-type 8 -j DROP
-A INPUT -j LOG --log-prefix "IPTABLES-DROPPED:"
-A INPUT -m mac --mac-source 00:1a:2b:3c:4d:5e -j DROP
-A OUTPUT -p tcp -m tcp --dport 8080 -j DROP
-A OUTPUT -d 157.240.8.35/32 -j DROP
(mubeen@kali)-[~]
$
```

3.14 Flush IPTABLES Firewall Chains or Rules

This command is used for maintenance and testing. Flushing a chain removes all the rules within it, effectively resetting that part of the firewall. This is a critical command to know for recovering from a misconfiguration or for starting a new ruleset from scratch.

```

(mubeen@kali)-[~]
$ sudo iptables -F

(mubeen@kali)-[~]
$ sudo iptables -L -n -v
Chain INPUT (policy ACCEPT 2288 packets, 416K bytes)
pkts bytes target      prot opt in      out     source      destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in      out     source      destination

Chain OUTPUT (policy ACCEPT 909 packets, 96258 bytes)
pkts bytes target      prot opt in      out     source      destination

```

3.15 Disable Outgoing Mails through IPTABLES

This rule enforces a security policy by preventing the system from sending outgoing email messages. This is a common control on user workstations or servers that should not be acting as mail clients to prevent data exfiltration or the unauthorized use of mail services.

```

(mubeen@kali)-[~]
$ sudo iptables -A OUTPUT -p tcp --dport 25 -j DROP

(mubeen@kali)-[~]
$ sudo iptables -L OUTPUT -n -v
Chain OUTPUT (policy ACCEPT 942 packets, 99756 bytes)
pkts bytes target      prot opt in      out     source      destination
 0    0 DROP      tcp  --  *      *      0.0.0.0/0    0.0.0.0/0    t
cp dpt:25

```

- -A OUTPUT: Appends the rule to the OUTPUT chain (for traffic originating from this machine).
- -p tcp: Specifies the TCP protocol, which is used by SMTP.
- --dport 25: Matches packets where the destination port is 25, the default port for the SMTP protocol.
- -j DROP: Silently drops all matching outgoing packets. Any application trying to send mail will fail to establish a connect

4.0 Challenges faced

4.1 Challenges

Rule Ordering: The order of rules in a chain is critical. A common mistake is placing a broad ACCEPT rule before a specific DROP rule, rendering the DROP rule ineffective. This was managed by frequently listing rules with `sudo iptables -L --line-numbers` to review their order and using `sudo iptables -I [CHAIN] [number]` to insert rules in the correct position.

Testing and Verification: Simply adding a rule did not guarantee it worked as intended. Each rule required active testing—using ping, nmap, curl, or attempting connections from other machines—to verify the firewall was behaving as expected.

5.0 Conclusion

This lab provided essential, hands-on experience in configuring a Linux firewall using IPTABLES. Through practical exercises, I gained proficiency in writing rules to filter traffic based on IP, port, protocol, and MAC address, while also implementing advanced features like rate limiting and connection tracking. The importance of rule order and testing was emphasized, preventing potential misconfigurations like self-lockout. These skills are directly applicable to real-world tasks such as securing servers, mitigating attacks, and enforcing access policies. Ultimately, this exercise reinforced how a firewall technically enforces a predefined security strategy, highlighting the critical link between policy and practical implementation in maintaining network security.